

Assignment 3

Group I
2026

Participating Members:

- Gerald Enright 577830
- Keletso Marota 601632
- Mogau Malope 600192

Question 1

1.1 QA, QC, Verification and Validation

Four separate tasks must be covered by a quality approach. **Quality control** is product-oriented and investigative, **quality assurance** is process-oriented and preventative, **verification** is concerned with whether the product was constructed correctly in accordance with its specifications, and **validation** is concerned with whether the correct product was constructed for its intended purpose. They are not interchangeable: validation by itself may fail to identify internal flaws, while verification by itself may faithfully implement an incorrect specification. Additionally, the labels are not as trustworthy as they often are; Trautsch, Herbold, and Grabowski (2020) discovered that the traditional definitions of test levels are out of step with current developments. Therefore, the strategic challenge is not which activity is relevant, but which combination produces sufficient evidence for a given risk.

1.2 Risk-Based Verification: What Earns Depth

Suites outgrow the time and money available, therefore choosing which tests to run at all is an ongoing research problem because verifying everything is not an option. In their evaluation of machine-learning techniques for test case selection and prioritization, Pan et al. (2022) define it as integrating incomplete and flawed test case data into models that forecast which are most worthwhile to run. The same three factors influence the decision whether it is made by the team or by an automated system:

- **Likelihood**: raised by complexity, concurrency, rate of change and defect history.
- **Impact**: raised by data corruption, exposure of personal data, loss of an accountability record or legal consequence.
- **Business or technical criticality**: raised when the function sits on a core journey, other components depend on it, or no workaround exists.

Team interpretation: Detectability should be taken into consideration as a fourth factor. Since a quiet failure, like a lost update or an authorization gap, can last indefinitely, it deserves more consideration than likelihood and impact alone would indicate.

Limitation: According to Pan et al. (2022), any method must be justified locally rather than imported because findings from prioritization studies are difficult to compare and significantly dependent on context. Additionally, any rating inherits the team's blind spots and becomes stale as the system changes.

1.3 Comparing Complementary Forms of Verification Evidence

Form of evidence	Strong at exposing	What it cannot prove on its own
Static analysis and peer review	Risky patterns in the artefact itself, missing checks, unsafe constructs and defects in requirements and design before code exists	Run-time behaviour; it is blind to configuration, real data and business-rule correctness
Unit tests	Logic defects inside one component, with the fastest and most precise fault localisation	That components work together; heavy mocking keeps a suite green while the assembled system fails
Integration / API tests	Contract and interaction failures across a boundary, including access decisions taken at the endpoint	The end-user outcome; its meaning depends entirely on representative environments and data
End-to-end and performance tests	Failures that appear only in the assembled system, or only under real volume and concurrency	Whether a journey is understandable; both are slow, brittle and prone to flakiness

There is a warning in the comparison. New definitions are required, according to Trautsch, Herbold, and Grabowski (2020), who discovered that neither unit nor integration tests were consistently better at identifying specific problem types in contemporary Java applications. Therefore, rather than making assumptions based on the test level name, evidence should be chosen based on the particular risk.

1.4 Automation, Quality Gates and Their Limits

Rather than insight, automation contributes repeatability: the same checks are performed on each modification, and the outcomes are preserved artifacts that may be linked to a progression choice. That evidence becomes consequential, a predetermined pass/fail condition in the pipeline, when a quality gate is used, and both its value and its risk come from the same source: Only what it encodes is asserted by a gate. The signal may not be audible. According to Parry et al. (2021), 59% of the developers they polled deal with flaky tests on a monthly, weekly, or daily basis, which directly reduces the degree to which a suite's conclusion can be accepted. Flaky tests are tests that fail inconsistently without any modification to the code under test. Conversely, coverage assesses execution as opposed to assertion. Once a threshold becomes a target, it is often met in the most economical manner possible. A gate is proof that certain requirements were met, never that they were sufficient.

1.5 Critical Question

Why is “all automated tests passed” insufficient evidence, by itself, to conclude that a software product is high quality or ready for release?

1. **It is a statement about a sample.** Because it is impossible to run everything, suites have already been chosen and prioritized (Pan et al., 2022), so a pass describes the behavior sampled rather than the behavior that really occurs.
2. **It covers only encoded expectations.** The most detrimental flaws are in actions that no one could have predicted, such as an uncaptured requirement, an unhandled condition, or a missing authorization check.
3. **Verification is not validation,** and the test level is not guaranteed by the test level (Trautsch et al., 2020). Green can only attest to the incorrect product's proper construction.
4. **The signal itself may be unreliable.** Most developers frequently encounter flaky tests (Parry et al., 2021), unrepresentative data and skipped or quarantined tests remain undetectable in a pass/fail light. The suite does not address usability or other quality issues.

1.6 Risk-to-Verification Evidence Map

Figure 1.1 Risk-to-Verification Evidence Map: tracing risk to evidence to a progression decision

1. Risk or quality concern	2. Why it deserves deeper checking (likelihood, impact, criticality)	3. Verification evidence chosen	4. What this evidence still cannot show	5. Progression or quality gate decision
R1 Two staff members claim the same record at once and one update is quietly overwritten, damaging data integrity.	Likelihood M · Impact H · Criticality H. It corrupts the ownership record that later work and reporting rely on. Rating: HIGH depth.	Primary: integration and API tests that fire concurrent calls at a real database. Supporting: static analysis of transaction boundaries, plus one targeted regression case.	The tests ran at test scale on seeded data, so real contention volumes and timing stay unproven.	GATE: do not progress until a repeatable concurrency case exists and visibly fails once the control is taken away.
R2 One user retrieves another user's record because object level authorisation is broken, exposing private data.	Likelihood M · Impact VH · Criticality VH. Personal data, legal and reputational fallout, and the failure happens silently. Rating: HIGHEST depth.	Primary: API tests for every role, including negative cases and forced browsing. Supporting: design review of the access rule and static analysis for absent checks.	Only the endpoints and roles somebody thought to test are covered, so every new endpoint starts out untested.	GATE plus a standing rule: each new endpoint ships with a negative authorisation test, and an untested endpoint counts as unproven rather than safe.
R3 List and oversight views slow down as the volume of data and the number of users grows.	Likelihood H · Impact M · Criticality M. The system keeps working, but staff reject it in daily use. Rating: MEDIUM to HIGH depth.	Primary: a performance and load test at a declared data volume and concurrency level. Supporting: query level measurement, and monitoring of real response times after release.	A synthetic load profile is not the real usage mix, and clearing a threshold once says nothing about sustained behaviour.	CONDITIONAL: progress on a recorded measurement against a declared threshold, carrying a stated duty to keep monitoring after release.
R4 A refactor quietly breaks a workflow rule or drops an audit entry.	Likelihood H · Impact H · Criticality H. Change is frequent, and the loss destroys accountability evidence. Rating: HIGH depth.	Primary: an automated regression suite at unit and integration level, run on every change. Supporting: peer review and static analysis.	Regression tests guard only behaviour already written down. They say nothing about new behaviour, or about tests that are skipped or unreliable.	GATE: a failing suite blocks the merge, and suite health (skipped, quarantined and unreliable counts) is reviewed, not just the pass or fail light.
R5 A first time user cannot finish the core task without help.	Likelihood M · Impact H · Criticality H. A public facing service that nobody understands is a service nobody adopts. Rating: HIGH depth.	Primary: validation with representative users in an observed task session, which is validation rather than verification. Supporting: a full journey walkthrough proving the path is reachable, and an accessibility check.	Automation proves the path can be reached, never that it makes sense to a person, and one small session does not generalise.	NOT AUTOMATABLE: progression needs recorded human validation evidence. A green pipeline adds nothing to this row.

Risk to Verification Evidence Map. Five example risks, each traced across five columns: the risk or quality concern, why it deserves deeper checking, the verification evidence chosen, what that evidence still cannot show, and the progression or quality gate decision it informs.

Question 2

2.1 Threat modelling: purpose and method

Threat modelling exists to surface plausible misuse before code is written, so that security requirements are derived from anticipated attacker behavior rather than discovered after the fact by a scanner or, worse, an incident. Shevchenko (2018) describe threat modelling methods as building an abstraction of the system together with a profile of potential attackers and a catalogue of resulting threats, which security requirements can then be derived from directly.

The method selected for this analysis is **STRIDE**, developed at Microsoft. STRIDE walks a data-flow diagram of the system and asks, for each component, whether it is exposed to Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, or Elevation of privilege. STRIDE is useful specifically because it gives engineers and security reviewers a shared, repeatable vocabulary for reasoning about a design before implementation begins.

STRIDE's value is not merely descriptive. Scandariato (2015) conducted one of the few empirical evaluations of STRIDE in practice, involving 57 final-year computer science students, and measured its cost and effectiveness directly: valid threats identified per hour, the rate of false positives, and the rate of false negatives. This is an important citation for a research brief specifically because it demonstrates that STRIDE is a useful structured method and an imperfect one.

2.2 Four security concern chains

Four concern areas are analyzed below, each tracing the required chain: plausible threat or misuse -> security requirement/control objective -> engineering control -> verification evidence. These four are drawn from the module's list of seven candidate areas and were selected because they apply directly to any modern web-based information system handling user accounts and user-submitted data, independent of CivicConnect's

specific architecture.

Authentication

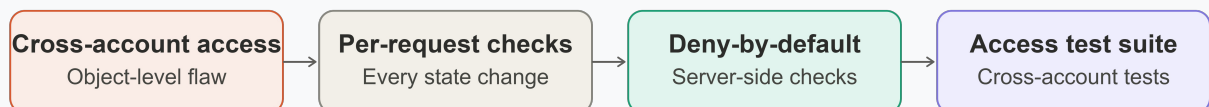
A weak or reused password creates a credential-stuffing attack surface, where lists of previously breached credentials are tried against a login form until one succeeds, leading to account takeover. The resulting requirement is that authenticator strength should be matched to the sensitivity of the action being protected, rather than applying one uniform login mechanism to every account regardless of risk. NIST SP 800-63B (Grassi, 2017) is the standards-level reference here: it defines three authenticator assurance levels and sets technical requirements for each, and has separately shifted authentication guidance away from forced periodic password rotation toward practices such as checking new passwords against known-breached password lists. The corresponding engineering control is login lockout and throttling calibrated to the assurance level required, and the verification evidence is a set of login-lockout test logs generated by simulating repeated failed attempts and confirming the throttling behaves as specified.



Weak login reuse {leads to takeover} -> Risk-matched login {Matched to risk} -> Login lockout {Throttled attempts} -> Lockout test logs {Simulated attacks}

Authorization / access control

A broken object-level access control flaw allows one authenticated user to view or modify another user's data purely by altering an identifier in a request, without the system checking whether that identifier belongs to the requester. The security requirement is that authorization must be verified on every request, not inferred once at login and then assumed to hold for the rest of a session. The engineering control is a deny-by-default, server-side authorization check applied to every state-changing endpoint, and the verification evidence is an automated test suite that deliberately attempts cross-account access and asserts that each attempt is rejected.



Cross-account access {Object-level flaw} -> Per-request checks {Every state change} -> Deny-by-Default {Server-side checks} -> Access test suite {Cross-account tests}

Secrets / configuration management

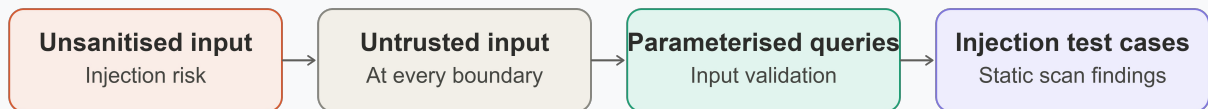
Credentials, API keys, or connection strings committed into source code or left in a default configuration file create a threat that does not require breaching the running system at all — only reading the repository. Secrets of this kind are typically introduced for development convenience, are difficult to rotate once embedded, and are frequently exposed later through version-control history, backups, or logs, which is why the requirement is that secrets must never be resolvable from the codebase in the first place, rather than merely “not visible” in the current file state. The engineering control is externalized secret storage — environment-injected configuration or a dedicated secrets manager rather than literal values in code — and the verification evidence is a clean, continuously enforced secret-scanning history in the CI pipeline.



Secrets in code {Committed credentials} -> No secrets in repo {Kept out of code} -> Secret manager {Environment injected} -> Secret scan history {Clean scan results}

Input / data handling

Unsanitized input accepted from a user-facing form or API field creates an injection attack surface - SQL, NoSQL, or command injection - where attacker-supplied text is interpreted as executable logic rather than as data. The requirement is that all input crossing a trust boundary is treated as untrusted regardless of its apparent source, including input from authenticated users. The engineering control is parameterized queries or an ORM layer combined with input validation at the boundary, and the verification evidence is a combination of static analysis findings targeting injection-pattern weaknesses and test cases that deliberately attempt injection payloads.



Unsanitized input {Injection risk} -> Untrusted input {At every boundary} -> Parameterized queries {Input validation} -> Injection test cases {Static scan findings}

2.3 Threat-to-Control Traceability Table

Threat / misuse	Security requirement or control objective	Possible engineering control	Verification evidence	Residual risk / limitation
Credential stuffing leading to account takeover	Authenticator strength matched to the risk of the protected action	Assurance-level-appropriate authentication with lockout and throttling	Login lockout test logs from simulated repeated-attempt attacks	Does not protect against credentials already compromised through an unrelated breach
Broken object-level access control (cross-account data access)	Authorization verified on every request, not assumed from a prior login	Deny-by-default, server-side authorization checks on all state-changing endpoints	Automated test suite that attempts cross-account access and asserts rejection	A newly added endpoint can omit the check unless enforcement is structural rather than per-endpoint
Secrets committed to source code or default configuration	Secrets never resolvable from the repository itself	Environment-injected configuration or a dedicated secrets manager	Clean, continuous secret-scanning history in CI	A scanner only detects patterns it is configured to recognize, leaving scope for undetected formats
Unsanitized input enabling injection (SQL/NoSQL/command)	All external input treated as untrusted at every trust boundary	Parameterized queries/ORM use plus input validation at the boundary	Static analysis findings for injection-pattern weaknesses plus targeted injection test cases	Static analysis coverage varies by language and framework, and cannot prove the absence of every injection path

2.4 How secure design, secure coding, and dependency/security checks complement one another

These three practices catch different classes of failure and none substitutes for the others. Dependency and software composition analysis (SCA) checks only ever detect known vulnerabilities in third-party code already catalogued against sources such as the National Vulnerability Database, they cannot see a flaw that exists only in first-party code, because there is no dependency to scan. Secure coding practice addresses that gap by catching classes of first-party mistakes - such as unsanitized input or hard-coded secrets - before any vulnerability record exists for them. Secure design, expressed through a threat model such as the STRIDE analysis above, catches a further class of flaw that neither of the other two can reach: a structural weakness in how the system is arranged, which exists before a single line of code is written and therefore cannot be “scanned” in the conventional sense. A system can pass every dependency scan and every static analysis check and still be insecure because its access-control model was never designed to prevent the misuse in question.

2.5 Two limitations of relying only on automated security tools or AI-generated security recommendations

Automated scanners disagree with one another, and coverage is tool-dependent. Imtiaz (2021) conducted a comparative empirical study of software composition analysis tools and found meaningful differences in the vulnerabilities each tool reported for the same codebase, concluding that the choice of tool materially affects what a team believes about its own dependency risk. A “clean” scan result is therefore a claim about that specific tool’s coverage, not an absolute statement about the code.

AI-generated code and AI security suggestions introduce their own, separately measured risk, and can create false confidence. Pearce (2025), in the most widely cited empirical study of GitHub Copilot, found that generated code suggestions contained exploitable vulnerabilities in roughly 40% of cases, and that a vulnerable suggestion was about as likely to be Copilot’s top-ranked choice as a secure one - meaning the flawed option is frequently the default rather than a rare edge case. A related user study by Perry (2023) found that developers using AI coding assistants produced significantly less secure code than those who did

not, while also rating their own solutions as more secure than the group not using AI assistance. Together, these findings indicate that an AI-generated recommendation - whether it is a code suggestion or a security opinion - carries risk that is not reduced simply because the output is confident or fluent, and must be independently verified rather than accepted as evidence.

2.6 Critical question

If a security scanner reports no high-severity findings, what important security claims may still remain unproven?

A clean scanner result only demonstrates the absence of the specific, cataloged patterns that particular tool is built to detect, within the scope it actually covers. It says nothing about business-logic flaws such as broken authorization, where the code may be syntactically and structurally unremarkable but still permits an action it should not - this class of flaw requires understanding what a request is supposed to be allowed to do, which a pattern-matching scanner cannot infer. It also says nothing about vulnerabilities outside the scanner's rule set, misconfigurations in the deployment environment rather than the code itself, or weaknesses introduced by components the scanner does not analyze. Finally, as the SCA comparison study above shows, "no findings" is itself tool-dependent: a different scanner applied to the same code may report differently, so the claim "the scan was clean" is bounded by which scanner ran it, not an absolute statement about the system's security.

Question 3

3.1 Environment Strategy

CivicConnect's Assignment 2 research considered a UI, backend/API and PostgreSQL database as the proposed architecture, subject to the team's later confirmation of the relevant decisions. As the project moves toward production, the team should keep separate development, testing/staging and production environments, and keep them as similar as possible so that a change verified in one environment can be trusted in the next.

Uncontrolled differences between environments, such as different dependency versions, different configuration values, or a staging database that is much smaller and cleaner than production, create deployment risk because a change can pass every check in development and still fail once it meets real production conditions. Configuration should be kept out of the source code and version-controlled separately for each environment rather than hard-coded, and staging should be built through the same automated process as production rather than configured by hand (Saleh, Madhavji and Steinbacher, 2025).

3.2 Secrets and Production Configuration

Database passwords, API keys and similar values should never be stored in source code, because anyone with repository access, including CI logs, could then reach production systems. The team's Assignment 2 research recommended using GitHub Actions' encrypted secrets rather than committed configuration files, subject to later confirmation of the relevant decisions.

GitHub Actions itself still needs to be managed carefully. Research on real GitHub CI workflows found recurring problems such as excessive workflow permissions and secrets being exposed through logs or third-party actions (Koishybayev et al., 2022). Protecting secrets is therefore an ongoing process rather than a one-off setting: CivicConnect should combine least-privilege access, secret scanning of the repository and regular secret rotation instead of treating encryption alone as sufficient.

3.3 Release and Deployment

A successful build does not automatically mean the system is ready for users. Continuous delivery is about keeping software in a state that could be released at any time, while continuous deployment goes further and releases that build to users automatically. Both rely on a pipeline that can repeatedly build, test and package a change without manual intervention (Shahin, Babar and Zhu, 2017).

Research on architecting for continuous delivery also shows that the ability to deploy, and to recover from a failed release, needs to be designed into the system and pipeline rather than added afterwards (Shahin et al., 2019).

For CivicConnect, the deployment stage could build on the GitHub Actions build and test approach recommended in the team's Assignment 2 research, subject to later confirmation of the relevant decisions. It should be automated and repeatable, with a clear procedure for what happens if a deployment fails partway. Staged rollouts and feature flags could reduce release risk further, but whether either is proportionate for CivicConnect is a decision for M3, not something this research assignment can settle.

3.4 Failure and Recovery

Application code is usually easier to roll back than data. An empirical study of ten large open-source systems found that database schemas change frequently and that each schema change typically forces a matching change in the application code that reads and writes it (Qiu, Li and Su, 2013). A schema migration that is not backward-compatible can therefore leave the previous version of the application unable to run against the new database structure, which removes the option of a clean rollback exactly when it is needed most.

For CivicConnect, the request and assignment table migrations discussed in Assignment 2 should be checked for backward compatibility, and the rollback path should be tested before an incident happens rather than during one. The team should also keep a documented backup and restore process with recovery time and recovery point targets, and should not treat a backup as reliable until a restore from it has actually been tested.

3.5 Operations and Observability

Functional testing shows that a feature works once, under the conditions that were tested; it does not show whether the team can detect or explain a problem that only appears in production. Observability is normally built from three types of information: logs, metrics and traces (Li et al., 2022). Logs explain what happened at a specific point, metrics show whether the system is healthy over time, and traces show where a request spent its time across components once the architecture is distributed enough for that to matter.

Collecting this data is not enough on its own; the team also needs to know what it means. Alerts should focus on problems that are significant and actionable rather than every small change, otherwise the team can end up ignoring them (Puli, 2025). For CivicConnect, a dashboard is only useful if it is tied to a clear target, for example that requester-facing status updates should reliably reach the portal within the 15-second window required by NFR-002 (A2, Task 3).

3.6 Operational Quality

Reliability, performance, scalability, cost and supportability are connected and should not be treated as separate checklist items. A service-level objective turns a general reliability goal into a measurable target, together with an error budget the team can use to decide whether to prioritise new features or stability work in a given period (Puli, 2025).

Performance and scalability claims need evidence from conditions that resemble real production traffic, not just correctness for a single test user, since an architecture that satisfies functional requirements does not automatically satisfy non-functional requirements such as response time under concurrent load (Shahin et al., 2019). Cost and supportability are easy to leave out of a research-stage discussion, but they matter in practice: a system that is reliable and fast but too expensive to run, or too complex for the team to operate and support, is not genuinely production-ready even if it passes every functional and load test.

Production-Readiness Evidence Matrix

Concern	Risk if ignored	Evidence required before release	Evidence observed after release
Environment strategy and configuration parity	Changes may work in development but fail in production.	Development and staging environments are documented and kept close to production.	No unexpected environment-specific failures after deployment.
Secrets and production configuration handling	Credentials or sensitive configuration may be exposed.	Secrets are stored outside source code and access is restricted.	Secret scanning and access reviews show no unexpected exposure.
Release and deployment automation	Manual deployment can introduce inconsistent or missed steps.	A repeatable build, test and deployment pipeline has been tested.	Deployment records show successful and repeatable releases.
Database migration compatibility and rollback	A failed migration may prevent the previous application version from working.	Migration and rollback procedures are tested against representative data.	Failed changes can be recovered without losing required data.
Backup and disaster recovery	Data may not be recoverable after a serious failure.	Backup, restore and recovery procedures are documented and tested.	Restore tests and recovery records demonstrate that backups remain usable.
Observability, logs, metrics, monitoring and alerts	Problems may occur without the team knowing what happened.	Relevant logs, metrics and alerts are configured and tested.	Monitoring shows system health and alerts identify significant problems.
Operational quality: reliability, performance and scalability	The system may meet functional requirements but fail under real usage.	Reliability and performance targets are defined and tested under representative load.	Production measurements are compared against the defined targets.

3.7 Critical Question

Why can software that passes functional tests and runs successfully on a developer's machine still be unready for production?

Functional tests normally confirm that the expected behaviour works under the conditions that were tested. They do not necessarily show what will happen when the system is deployed into a different environment, receives concurrent requests, loses a dependency, or needs to recover from a failed deployment.

A developer's machine can also have different configuration, dependency versions, database data and permissions from the production environment. Without deployment checks, monitoring, recovery procedures and performance evidence, a system can therefore appear to work correctly while still being difficult to operate or recover in production.

Question 4

Research finding	Engineering concern it addresses	Candidate approach / evidence to consider	Trade-off / limitation found in research	Decision M3 must still make
Running every test is infeasible, so selection and prioritization is itself an engineering decision.	Finite verification effort against uneven product risk	A documented scheme ranking features and quality concerns by likelihood, impact and criticality, with a depth of evidence per band	Results across prioritization studies are hard to compare and context-dependent. Ratings inherit blind spots and go stale	Which CivicConnect requirements are risky enough to justify deeper evidence, on what scale, and who ratifies the rating?
Automated suites frequently contain	Trustworthiness of the automated evidence	Tracking flaky-test rate and fixing or	Parry et al. (2021) found most developers	How will CivicConnect detect and track flaky

flaky tests that fail without any code change, which erodes confidence in a pass/fail signal.	used at a quality gate	quarantining flaky tests before treating a suite result as gate evidence	deal with flaky tests regularly; quarantining a test also removes its coverage, trading signal reliability against coverage	tests, and what flakiness threshold disqualifies a test from gating a release?
Threat modelling (e.g. STRIDE) surfaces plausible misuse before code is written, but empirical evaluation shows it still produces measurable false positives and false negatives even when applied correctly.	Design-stage identification of security requirements before implementation	A structured threat-modelling session run against CivicConnect's actual data-flow diagram, with resulting threats tracked to specific requirements	Scandariato et al. (2015) measured meaningful false-positive and false-negative rates among trained analysts, so a threat model is a starting point rather than a complete threat inventory	Which CivicConnect components or data flows warrant a full threat-modelling pass, and who reviews the resulting threat list before it becomes a requirement?
Automated security scanners disagree with each other on the same codebase, and AI-generated code or security suggestions carry a separately measured, non-trivial vulnerability rate.	Over-reliance on a single automated or AI-generated signal as proof of security	Combining more than one verification source (e.g. dependency scanning plus manual/secure-coding review) rather than treating one clean scan as sufficient evidence	Imtiaz et al. (2021) found scanner results vary by tool; Pearce et al. (2025) and Perry et al. (2023) found AI-suggested code is frequently insecure and can create false confidence in the developer using it	Which combination of tools and manual review will CivicConnect require before a security-relevant change can be merged or released?
Uncontrolled differences between development, staging and production environments allow a change to pass every pre-release check and still fail once it meets real conditions.	Deployment reliability across environments	Keeping environments as close as possible (dependency versions, configuration approach, data shape) and building staging through the same automated process as production	Full parity has a cost in infrastructure, data volume and time, and is rarely complete in practice, so some difference must be accepted rather than assumed away	What level of environment parity will CivicConnect actually maintain, and which differences, if any, are accepted as a known risk?
Database schema changes are frequently coupled to matching application-code changes, so a non-backward-compatible migration can remove the option of a clean rollback exactly when it is needed.	Failure and recovery capability during a release	Testing migration and rollback paths against representative data before release, and checking that new migrations remain backward-compatible with the previous application version	Qiu, Li and Su (2013) studied established open-source systems; the same coupling pattern may look different in CivicConnect's specific schema, and rollback testing itself needs time and representative data a small team may not always have ready	Which CivicConnect migrations are treated as high-risk enough to require a tested rollback path before release, and what recovery-time/recovery-point targets are acceptable?

References

- Grassi, P.A., Garcia, M.E. and Fenton, J.L., 2017. Digital Identity Guidelines (翻譯版). NIST special publication, 800, pp.63-3.
- Imtiaz, N., Thorn, S. and Williams, L., 2021, October. A comparative study of vulnerability reporting by software composition analysis tools. In Proceedings of the 15th ACM/IEEE international symposium on empirical software engineering and measurement (ESEM) (pp. 1-11).
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B. and Karri, R., 2025. Asleep at the keyboard? assessing the security of github copilot's code contributions. Communications of the ACM, 68(2), pp.96-105.

- Perry, N., Srivastava, M., Kumar, D. and Boneh, D., 2023, November. Do users write more insecure code with ai assistants?. In Proceedings of the 2023 ACM SIGSAC conference on computer and communications security (pp. 2785-2799).
- Scandariato, R., Wuyts, K. and Joosen, W., 2015. A descriptive study of Microsoft's threat modeling technique. *Requirements Engineering*, 20(2), pp.163-180.
- Shevchenko, N., Chick, T.A., O'riordan, P., Scanlon, T.P. and Woody, C., 2018. Threat modeling: a summary of available methods (No. AFLCMCAZS).
- Pan, R., Bagherzadeh, M., Ghaleb, T.A. and Briand, L. (2022) 'Test case selection and prioritization using machine learning: a systematic literature review', *Empirical Software Engineering*, 27(2), article 29.
- Parry, O., Kapfhammer, G.M., Hilton, M. and McMinn, P. (2021) 'A survey of flaky tests', *ACM Transactions on Software Engineering and Methodology*, 31(1), pp. 1-74.
- Trautsch, F., Herbold, S. and Grabowski, J. (2020) 'Are unit and integration test definitions still valid for modern Java projects? An empirical study on open-source projects', *Journal of Systems and Software*, 159, article 110421.
- Koishybayev, I., Nahapetyan, A., Zachariah, R., Muralee, S., Reaves, B., Kapravelos, A. and Machiry, A. (2022) 'Characterizing the security of GitHub CI workflows', in *31st USENIX Security Symposium*, pp. 2747-2763.
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J. and Liu, X. (2022) 'Enjoy your observability: an industrial survey of microservice tracing and analysis', *Empirical Software Engineering*, 27(1), article 25.
- Puli, B. (2025) 'Site reliability engineering (SRE) and observations on SRE process to make tasks easier', *arXiv preprint*, arXiv:2505.01926.
- Qiu, D., Li, B. and Su, Z. (2013) 'An empirical analysis of the co-evolution of schema and code in database applications', in *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2013)*, pp. 125-135.
- Saleh, S.M., Madhavji, N. and Steinbacher, J. (2025) 'A systematic literature review on continuous integration and deployment (CI/CD) for secure cloud computing', *arXiv preprint*, arXiv:2506.08055.
- Shahin, M., Babar, M.A. and Zhu, L. (2017) 'Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices', *IEEE Access*, 5, pp. 3909-3943.
- Shahin, M., Zahedi, M., Babar, M.A. and Zhu, L. (2019) 'An empirical study of architecting for continuous delivery and deployment', *Empirical Software Engineering*, 24(3), pp. 1061-1108.